RAG Debugging — Worked Example: Solution

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Hints

Story — `rag_debugging_practice.py`: RAG bugs rarely crash outright — they usually just give quietly worse answers, which makes them look like "the model is confused" when the model actually did nothing wrong. Each round's fix sits right next to a test that checks the retrieval side directly, with no model call needed. **If not:** you'd keep rewriting prompts for problems that actually live in chunking, embeddings, or shared state — places a prompt rewrite can never fix.

Every fix and test below goes in `practice/rag_debugging_practice.py`, and runs with `pytest rag_debugging_practice.py -v` from inside `practice/`. None of the tests need an API key.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent
- Real cause vs. symptom fix

Round: Basic

Real cause: `text[start:end - 1]` — the `- 1` is an off-by-one slicing mistake. Python slicing (`text[start:end]`) already stops one character before `end`; subtracting 1 again drops one more character from every chunk — here, the space at each boundary.

Story: no error anywhere, just slightly wrong data — this is the single most common kind of RAG bug. This round trains actually checking a chunker's output against its original input character for character, not just confirming "did it run without crashing". **If not:** one dropped character per chunk would quietly damage every single document you ever index, with nothing ever flagging it.

The fix:

```
def chunk_by_chars(text, chunk_size):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        # why: text[start:end] already stops before end — no "- 1"
```

```
        chunks.append(text[start:end])
        start = end
    return chunks
```

Test that would have caught it:

```
def test_chunk_by_chars_reproduces_the_original_text():
    text = "The quick brown fox jumps over the lazy dog"
    chunks = chunk_by_chars(text, chunk_size=10)
    # how: joined back together, the chunks must equal the input
    assert "".join(chunks) == text
```

"Joined chunks equal the input" is a strong, simple check for any chunker without overlap — it catches dropped characters, doubled characters and off-by-one errors in one line.

Round: Intermediate

Real cause: documents were embedded with one model (`text-embedding-3-small`) and questions are now embedded with a different one (`text-embedding-3-large`). Different embedding models don't share the same number space — comparing a `-large` question vector with `-small` document vectors compares two unrelated coordinate systems, so every score comes back low and meaningless. Nothing raises an error, because both models return valid-looking vectors — which is why this first looks like a generation problem, not a config mismatch.

Story: every question got worse all at once, right after a config change — "everything, suddenly" is a pattern that points straight at shared setup, not at any one specific question. This round trains checking what actually changed recently before reaching for the model as the suspect. **If not:** you'd end up tuning the prompt for a problem no prompt rewrite could ever fix.

The fix:

```
# why: ONE place names the embedding model, and both sides use it —
# they can't drift apart again
EMBED_MODEL = "text-embedding-3-small"

INDEX_EMBED_MODEL = EMBED_MODEL   # used by indexing.py
QUERY_EMBED_MODEL = EMBED_MODEL   # used by retrieve.py
```

If you ever do switch models, the index must be rebuilt with the new model too — old vectors can't be mixed with new ones.

Test that would have caught it:

```
def test_index_and_query_use_the_same_embedding_model():
    assert INDEX_EMBED_MODEL == QUERY_EMBED_MODEL
```

A one-line test, but it turns a silent quality drop into a loud, named failure the moment someone edits only one side.

Round: Real-world

Real cause: fixed-size chunking cut a sentence in half, right at the spot holding the fact the question needed. Retrieval technically "worked" — both halves came back — but the model never reliably joined "Customers" (end of one chunk) to "have 30 days..." (start of the next) as one sentence. The symptom shows up in generation; the cause is one step earlier, in chunking.

Story: only one question fails, and only because its answer happens to sit right on a chunk boundary by chance. This round trains deliberately constructing a test input that puts the answer exactly on a boundary on purpose, instead of hoping a random test case happens to hit it. **If not:** the bug would keep randomly hitting whichever facts happen to land on a boundary, with no test ever reliably catching it.

The fix:

```
def chunk_by_chars_with_overlap(text, chunk_size, overlap):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        # why: step back by `overlap`, so text near a boundary appears
        # whole in at least one chunk
        start = end - overlap
    return chunks
```

Splitting by paragraph (Doc08's `chunk_by_paragraph`) is the more thorough fix, since it cuts at natural breaks instead of at a character count.

Test that would have caught it:

```
def test_answer_on_a_chunk_boundary_is_still_whole_in_one_chunk():
    answer = "Customers have 30 days to request a refund."
    # how: 90 characters of filler put the answer across position 100
    document = "x" * 90 + answer + " " + "y" * 100
    chunks = chunk_by_chars_with_overlap(document, chunk_size=100,
                                         overlap=50)
```

```
    found_whole_sentence = False
    for chunk in chunks:
        if answer in chunk:
            found_whole_sentence = True
    assert found_whole_sentence
```

With no overlap, the answer is split at character 100 and the test fails; with 50 characters of overlap, one chunk holds the whole sentence. This is Doc08's own `chunk_boundary_split` exercise, turned into a test.

Round: Multi-agent

Real cause: the Research agent writes only a plain-text summary into shared state, dropping the score and source the retriever returned. The Analysis agent's "flag unsourced claims" check works exactly as designed — it just never receives the sources it needs, because they were thrown away one node earlier. Retrieval and generation were both correct; the bug is in what one agent chose to write into shared state.

Story: testing the retriever in isolation passes forever, because the retriever itself was never actually broken — the bug is downstream. This round trains testing what a node *hands on* to the next one, not just what it internally computes. **If not:** a correct, well-sourced fact would keep getting quietly softened into a vague, hedged answer, with every individual part of the pipeline still looking fine on its own.

The fix:

```
from typing import TypedDict
from langgraph.types import Command

class SharedState(TypedDict):
    task: str
    research_findings: str
    research_sources: list   # [{"text": ..., "score": ..., "source": ...}]

def retrieve(question, k):
    # stand-in for the real retriever from Doc08
    return [{"text": "Customers have 30 days to request a refund.",
             "score": 0.91, "source": "policy.md"}]

def summarize_findings(results):
    # stand-in for the Research agent's own summary call
    return "Customers can request a refund within 30 days."

def research_node(state):
```

```
    results = retrieve(state["task"], k=3)
    summary = summarize_findings(results)
    sources = []
    for result in results:
        sources.append({
            "text": result["text"],
            "score": result["score"],
            "source": result["source"],
        })
    # why: the sources travel WITH the summary, so the Analysis agent
    # can check where each fact came from
    return Command(goto="analysis_agent", update={
        "research_findings": summary,
        "research_sources": sources,
    })
```

Test that would have caught it:

```
def test_research_node_passes_sources_on_to_analysis():
    result = research_node({"task": "refund window"})
    sources = result.update["research_sources"]
    assert len(sources) == 1
    assert sources[0]["source"] == "policy.md"
```

The test checks the Research node's shared-state *update*, not whether the retriever found the right chunk — that's exactly the part that was broken.

Real cause vs. symptom fix

A symptom fix here would be adding "be precise about numbers" to the prompt (Round 1 or 3), lowering the similarity threshold until *something* comes back (Round 2), or telling the Analysis agent to trust every claim (Round 4). Each makes the visible symptom smaller while the real cause — a slicing bug, two embedding models, a sentence cut in half, sources thrown away — stays in place. In RAG, "the answer looks wrong" is almost always the last layer to show a problem that started earlier: in chunking, in embeddings, or in what gets handed from one step to the next. Check those first, with tests like the ones above, before you touch the prompt.